Individual Report

An LLM-Assisted Strategy Layer for a Real-Time Racing Simulator

Junjie Peng

Student Number: 2703712

sg25291@bristol.ac.uk

Department of Computer Science

University of Bristol

August 2026

Abstract

This project asked whether IBM Granite could advise an autonomous TORCS driver without being placed inside its real-time control loop. I took responsibility for the bot's technical foundation: the SCR protocol layer, the initial low-level controller and safety layer, the original track model, Granite integration, replay and latency-evaluation tools, HPC deployment, and the strategy information shown on the dashboard. My teammate later developed the driving-behaviour tuning, opponent avoidance, crash recovery, pit handling, and most of the final test suite; I refer to those parts only as context for the integrated system.

The defining constraint was timing. TORCS expects a command approximately every 20 milliseconds, while Granite took 6.7 to 7.6 seconds to respond in the final live tests. Rather than asking the model to drive the car directly, I separated the fast controller from the slower advisor. The controller never waits for Granite, and model advice must pass through a local safety layer before it can affect the car.

My evaluation process changed as the project developed. I began by comparing complete races, but found that this was slow and gave each prompt different situations. I therefore built an offline replay tool that presents prompts with the same recorded vehicle states. This report concentrates on the decisions behind that architecture and evaluation method, including two measurements I later rejected and an early interpretation of strategy diversity that I withdrew. The most important experiment still missing is a controlled comparison of the same controller with and without Granite advice.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

Junjie Peng

August 2026

Contents

Introduction and Context

Our group project connects TORCS 1.3.7, an open-source racing simulator, to IBM Granite through an OpenAI-compatible API [4]. The complete system combines a conversational racing engineer, live telemetry, event-driven commentary, and an autonomous bot that can receive strategic advice from Granite.

Within that system, I took responsibility for the bot's technical foundation and strategy pipeline. I implemented simulator communication, the initial local controller and safety layer, the original track model, Granite integration, replay and latency tools, HPC deployment, and the dashboard's strategy output. My teammate later concentrated on driving-behaviour tuning, opponent avoidance, crash recovery, pit handling, and most of the final pytest suite. Keeping that boundary explicit is important in an individual report, so I discuss those later features only where they affected integration.

The feature became interesting because its two components operate at incompatible speeds. TORCS expects steering and throttle approximately every 20 milliseconds; in my deployment, Granite normally needed several seconds. Work such as SayCan shows how a language model can provide high-level planning above lower-level skills [1], but placing such a model directly inside this control loop would cause the car to miss hundreds of simulator updates.

TORCS was useful for investigating this problem because it exposes detailed telemetry and supports repeatable tracks and vehicles. Its Simulated Car Racing Championship interface provides the sensor and actuator protocol used by the bot [2, 3]. The reference snakeoil client gave me a low-level starting point, but no mechanism for accepting slow, uncertain language-model advice.

My central design decision was therefore not to make Granite faster, but to prevent its speed from becoming the controller's problem. I built a two-clock architecture, then added replay and latency tools to test the slow side independently. This report follows that development in the order I experienced it: first the architectural constraint, then the evaluation instrument, and finally the measurements and conclusions that did not survive closer checking.

Example of Innovation and Creativity

I began with what seemed the most convincing test: give each prompt a complete race and compare the result. It quickly became clear that this was also the least controlled test. A race took roughly 10 to 15 minutes, and a small early steering difference could change the traffic, collisions, and tyre conditions encountered later. After several hours, I could still be comparing prompts that had never faced the same situation.

That experience changed the way I thought about the simulator. For the strategy layer, TORCS was mainly a source of vehicle states. The strategy code could not tell whether a state came from a live race or a file. I therefore recorded the inputs passed to Granite and prepared the `race2`, `race4`, `race4_lowfuel`, and `race5` JSONL corpora, containing 2,870 states in total.

I then developed `bot_replay.py` to run the strategy layer against those records without starting TORCS. This turned a prompt comparison from two unrelated race reports into a paired experiment. The `--limit` option uses deterministic, evenly spaced sampling, so two variants given the same trace and limit receive exactly the same selected states. Granite itself is still variable at temperature 0.2, and stronger claims would require repeated runs or confidence intervals, but at least the simulator input no longer moves between comparisons.

A smaller version of the same reasoning arose when the dashboard appeared to be frozen on `ATTACK`. Making Granite choose more varied strategies would have improved the display by changing the behaviour being measured. I treated it as a presentation problem instead and implemented `display_label()`. The same strategy can appear as Leading: holding pace, Chasing: full throttle, or Attacking, depending on race context. Tests confirm that computing the label does not alter the value passed to the controller. This was a modest feature, but it protected a boundary I considered important: presentation should explain a decision, not quietly change it.

Technical Development

Keeping a seven-second model out of a 20 ms loop

The feature rests on two timing domains, shown in Figure 1. The fast domain receives an SCR packet, updates the track model, and computes steering, throttle, brake, and gear. It is local, deterministic, and independent of the network. The slow domain periodically describes the race to Granite and reduces its reply to one of five model-selectable strategies: `ATTACK`, `NORMAL`, `DEFEND`, `SAVE_FUEL`, or `PIT`.

I kept `BLOCK` out of Granite's options. Blocking depends on the rapidly changing position of another car, so advice arriving several seconds later could be actively dangerous. This was an early example of a rule I used throughout the bot: allocate a decision according to how quickly it must be corrected, not according to how strategically important it sounds.

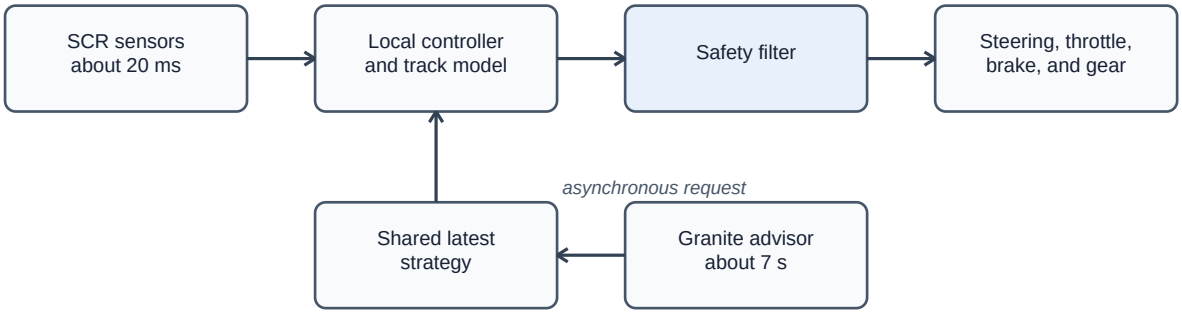

Figure 1: Separation of the fast controller and the slower Granite strategy service.

The essential rule is that the fast domain never waits for the slow one. Granite publishes its latest valid decision to a shared value; the controller reads that value when convenient. If the service is unreachable or its reply cannot be parsed, the car continues with the previous strategy. A stale strategy is usually manageable. A blocked control loop is not. This follows the broader separation between deliberative planning and reactive control discussed in embodied AI work [1], but the latency ratio here is unusually large.

Between the two domains, I implemented the initial `safety_filter`. Granite can request aggressive behaviour, but the local controller retains authority over the car's physical limits. My teammate later added pit- and opponent-aware rules to the final version; those extensions are not presented here as my individual work.

I placed the map trust checks in one narrow component rather than scattering them through the controller. Table 1 records the five conditions. This made the safety claim easier to state and, more importantly, easier to test.

#	Condition	Reason
1	Odometer calibrated after a practice lap	Before calibration, accumulated map alignment error may affect lookahead.
2	Car is on the racing line	The map describes the intended line rather than every possible off-line path.
3	No opponent within 100 m ahead	The track map does not represent traffic.
4	Sensor range agrees with the map prediction	A disagreement may indicate an obstacle or traffic not represented in the map.
5	Local speed limit is at least 150 km/h	Map assistance has more benefit on faster sections and less safety margin near an apex.

Table 1: Map trust conditions.

Turning prose into a control decision

The first practical difficulty was that Granite returns language, while the controller needs a value. Even when two replies express the same judgement, their wording can differ. I implemented `midware/bot_strategy.py` to assemble a prompt from speed, fuel, damage, track position, race position, and nearby gaps, then reduce the reply to a permitted strategy.

I made the parser tolerant about presentation but strict about output. It accepts several reasonable response formats, yet it publishes only a valid strategy value. If a reply remains ambiguous, the previous strategy is retained. Combined with the two-clock design, malformed prose therefore degrades to no new advice rather than a failed driving loop. The team suite covers prompt construction, parsing, safety integration, and failure handling; my own test work included updates to the Granite runtime and safety-integration tests.

Giving the controller a view beyond its sensors

The strategy layer was of little use if the local controller could not execute its advice safely. In early tests, the car carried too much speed into corners that were not yet clear in the short-range sensors. I developed the original `track_model.py` to represent the circuit as segments with curvature, length, and a derived speed limit. The controller could then brake for a corner before the sensors showed it clearly.

The initial speed calculation used a basic friction-circle approximation:

$$v = sqrt(a_{lat} R)$$

The first version used constant lateral acceleration. It kept the car safe, but made map-assisted driving unnecessarily slow in high-speed corners. I added a simplified downforce term, giving:

$$v^2 = a_{lat} R / (1 - k_{ero} a_{lat} R)$$

A second error came from treating adjacent same-direction segments independently. The straightening correction then assigned 155 km/h to one 80 m-radius corner instead of 120 km/h. I fixed this by merging those segments before applying the correction. Braking points were propagated backwards over two laps, and I calibrated the odometer against measured lap distance to reduce accumulated map error. My teammate later tuned the final corner-speed parameters and added further

chicane handling.

When inference became the bottleneck

Once the strategy layer worked, local inference became the bottleneck. Response times were long and varied enough to slow both development and measurement. I moved the service to Bristol's BluePebble cluster and wrote the Slurm scripts, preflight check, readiness polling, and connection helper needed to use it reliably. This infrastructure did not improve the strategy itself, but it replaced an uncertain manual process with a repeatable one and made the final latency comparison possible.

Evaluation Skills

What the measurements do---and do not---show

The final group suite contains 17 test files covering SCR communication, control behaviour, recovery, track lookahead, traffic, latency, and Granite integration. On 18 August 2026, `pytest tests/bot` reported 247 passing tests and 16 additional parameterised subtests. Most of that suite was built by my teammate. My own additions were narrower: the dedicated 113-line test file for `display_label()` and updates to the Granite runtime and safety-integration tests.

For that reason, Table 2 includes only measurements directly connected to my work. It does not use the team's recovery, pit, or fault-injection results as evidence of my individual contribution, and the figures should be read as measurements of the tested runs rather than general guarantees.

Measure	Result
Control computation	Median 0.018 ms for the measured local computation, against a 20 ms control-cycle target
Granite latency	Local LM Studio median 9.325 s; HPC 6.9 to 7.2 s; live-race samples 6.7 to 7.6 s
Replay corpus	2,870 states across race2, race4, race4_lowfuel, and race5

Table 2: Measurements related to my contribution.

Opponent avoidance, crash recovery, pit behaviour, and the majority of the final pytest suite were implemented and evaluated by my teammate. Their results are therefore not presented as evidence of my individual contribution.

Checking the instrument, not only the result

Because replay became the basis of my prompt comparison, I needed to test the instrument itself. I compared three live-race latencies---6.7, 7.2, and 7.6 seconds---with offline measurements of 7.0, 7.1, and 7.5 seconds on the same platform. The samples are small, but they gave me preliminary evidence that running TORCS did not add a large inference delay.

This check does not prove that replay reproduces a complete race. A live strategy changes the car and therefore the states encountered later. Replay is suitable for asking how two prompts respond to the same fixed situations; it is not a substitute for measuring driving performance. I also implemented `bench_latency.py` so model-server latency could be separated from simulator timing. Without that separation, a slow result could be blamed on the wrong component.

Discarding measurements that did not answer the question

The final platform test showed that two earlier measurements should not be used.

The first was the `sensitive` column in the prompt-comparison output. It was intended to count decisions that changed after a meaningful change in race state. However, the implementation also treated a change in the explanatory text as a change, even when the strategy remained the same. At temperature 0.2, wording can vary between calls. The perturbation also changed fuel from a high value to 60 L, which was still sufficient for the tested race. The test therefore did not create a state in which a different strategy was clearly expected.

The second was a note in `analyse_bot_trace.py` that described a 4 to 6 second polling cycle. This value had been hardcoded for an earlier configuration and no longer matched the final system. I documented both figures as values that should not be quoted.

Revising my own conclusion

The most uncomfortable result was the one that contradicted my earlier interpretation. I had described the legacy prompt as showing mode collapse because it produced only one distinct strategy on the selected corpus. When I later compared the variants on the same states, the reasoning prompt also returned `ATTACK` for 48 of 50 cases.

The simpler explanation was the data: the trace came from a trouble-free race where attacking was usually reasonable. Diversity was not the same as quality when the situations themselves were uniform. I removed the mode-collapse claim rather than defend a convenient metric, and this directly motivated the more discriminating corpus proposed in future work.

Project Management Skills

Building the feature from the bottom up

The group divided implementation by feature. I took the bot's technical foundation and strategy pipeline, which gave my work a natural dependency order. As Table 3 shows, I started with communication and control, then added the track and safety layers, and only then connected Granite. This bottom-up order meant that every new layer rested on something I could already run and inspect.

Phase	Period	Main work
1	June to 2 July	SCR parser, low-level controller, and pure-pursuit baseline
2	3 to 14 July	Main repository integration, safety filter, and track model
3	15 to 26 July	Granite strategy integration and prompt development
4	27 July to 18 August	Replay traces, display-label tests, reasoning-prompt analysis, and HPC deployment

Table 3: Personal development phases.

My weekly routine used Tuesday and Thursday afternoons for concentrated implementation. Monday and Wednesday were mainly used for integration and documentation, and Friday morning was used for review and meeting preparation. This schedule was adjusted when integration work or experiments required longer continuous sessions.

Using Git as an engineering record

I used Git for more than storing code. Routine commits marked small implementation steps, while commits that changed an experimental conclusion recorded the reason and the affected measurements. I worked on `dev/aibot` and integrated through pull requests, so an incomplete controller did not have to destabilise the group demonstration.

The commit history was useful, but it was not a replacement for evaluation notes. I kept separate documents for the HPC platform comparison and replay experiments, including the configuration, observations, and limitations. My teammate maintained the fault-injection records and the main bot test matrix.

Explaining decisions to the team and client

I contributed to weekly progress reports for IBM stakeholders, including updates on the Granite integration, middleware changes, and an early-August demonstration video. When presenting the strategy switch, I explained that it controls requests for new model advice but does not disable the local safety controller. This distinction was important because the dashboard could otherwise imply that Granite directly controlled the car. The client response to the demonstration was strongly positive.

Within the group, the clearest disagreement concerned the strategy badge. It appeared static when `ATTACK` remained appropriate for a long period, and changing the model output was suggested as a way to make it look more active. I argued that this would optimise the appearance of intelligence by changing an already measured decision. Rather than asking the team to accept that argument on trust, I proposed the display-label approach and added tests showing that a label cannot change the value returned by the safety filter.

Managing risk without stopping development

The main risks were simulator instability, model latency, and unsafe interaction between the two timing domains. I reduced the part I controlled through the non-blocking architecture, the initial safety layer, deterministic replay, and the move to BluePebble when local inference became too variable. My teammate led the later fault-injection and broader regression work. The HPC deployment took longer than I expected, but it kept the final prompt and latency tests from depending on an unstable local service.

Critical Self-Evaluation and Future Work

What I would do differently

My main evaluation mistake was collecting the data I could obtain before defining the cases that would answer the question. The recorded corpora mainly contain trouble-free races, precisely the situation in which several prompts can reasonably return `ATTACK`. If I repeated the project, I would begin by defining low-fuel, high-damage, and close-traffic cases, then collect or construct traces around them.

I also selected one metric because it was easy to compute. Counting distinct strategies did not measure whether the selected strategy was correct. A more useful metric would compare each decision with a defensible reference label, although producing those labels requires manual work and agreement on ambiguous cases. In future evaluations I would write down the expected interpretation

and failure mode of each metric before implementing it.

The largest omission is also the experiment a reader is most likely to ask for: the same controller, on the same tracks, with Granite enabled and disabled. I measured latency, prompt behaviour, replay behaviour, and platform differences, but not whether Granite improved lap time, consistency, damage, or off-track frequency. I spent the final week on HPC deployment and the platform comparison. In hindsight, I would run the ablation first.

I also spent too much time on infrastructure relative to the evaluation design. The BluePebble scripts were necessary for stable testing, but the work expanded because the tasks were concrete and easy to verify. Designing a useful corpus was less straightforward, but it was more important to the central research question. I should have reviewed that balance earlier with the supervisor.

Future work

The first priority is to build a strategy corpus containing states where the expected decision genuinely changes. Examples include crossing a fuel threshold, accumulating enough damage to justify conservative driving, and remaining in close traffic for several control cycles.

The second priority is the strategy-layer ablation described above. It should be repeated on the same tracks and controller configuration so that the effect of Granite can be separated from the quality of the local controller.

The `sensitive` metric should be replaced with a measure based on semantic equivalence and reference labels. An LLM-based judge could be explored, but its decisions should be checked against a manually labelled subset rather than treated as ground truth.

The track model also creates an opportunity to compensate for inference delay. Because Granite responds approximately seven seconds after a request, the prompt could describe a predicted future state rather than only the current one. This would require testing prediction error as well as strategy quality.

Finally, race traces provide pairs of states and model decisions. A smaller model could be trained on these data and compared with Granite. This would test whether the useful part of the strategy behaviour requires a general language model or can be reproduced with a faster specialised model.

Closing reflection

I expected the hardest part to be connecting a language model to a racing car. In practice, the harder problem was deciding what the model had contributed and building instruments capable of answering that question.

My technical contribution was the bot's foundation: the SCR protocol layer, the initial controller and safety layer, the original track model, Granite integration, replay and latency tools, HPC deployment, and dashboard strategy labels. Local control remained below the 20 ms target while multi-second model responses stayed outside the driving loop. The integrated bot's opponent handling, recovery, pit behaviour, later tuning, and most of the final regression coverage were completed by my teammate and are not claimed as my individual work.

The result is less tidy than a report of uniformly successful experiments. I rejected two measurements, withdrew the mode-collapse interpretation, and did not complete the most direct ablation. I regard that as a limitation of the project but also as evidence that the evaluation process changed my conclusions rather than merely confirming them. The current system is therefore best understood as a functioning basis for the next experiment: repeated, deliberately varied races with Granite enabled and disabled.

References

[1] Ahn, M. et al. (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. Conference on Robot Learning.

[2] Wymann, B., Espie, E., Guionneau, C., Dimitrakakis, C., Coulom, R. and Sumner, A. (2000). TORCS, The Open Racing Car Simulator. Available at: http://torcs.sourceforge.net.

[3] Loiacono, D., Cardamone, L. and Lanzi, P. L. (2013). Simulated Car Racing Championship: Competition Software Manual. arXiv:1304.1672.

[4] IBM Research (2026). Granite. Available at: https://research.ibm.com/topics/granite.